Dead Store Elimination Using Backward Data Flow Analysis

Yung-Chieh Chung
Student ID: R14173058
Advanced Compiler Design - Homework 6
National Taiwan University
CSIE 5054

***Abstract*—This paper presents an implementation of a Dead Store Elimination (DSE) optimization pass for LLVM IR using backward data flow analysis. The pass identifies and removes store instructions whose values are never read before being overwritten, thereby reducing memory bandwidth usage and instruction count. Our implementation achieves a 10.84% reduction in store instructions on benchmark code while maintaining semantic correctness through conservative analysis and explicit volatile store protection. The system is implemented in Rust using the inkwell library (LLVM 21.1 bindings) and includes comprehensive testing infrastructure with automated benchmarking.**

***Index Terms*—Dead store elimination, compiler optimization, data flow analysis, LLVM, program transformation**

I. Introduction

DEAD Store Elimination is a fundamental compiler optimization that removes unnecessary write operations to memory. A store is considered "dead" if it writes to a memory location that is overwritten before being read, if the memory location is never accessed after the store, or if the store would have no observable side effects. This optimization provides several benefits:

- Reduced memory bandwidth usage
- Decreased cache pollution
- Lower instruction count
- Improved overall program execution time

While LLVM includes its own DSE pass, this work implements a standalone optimization pass to demonstrate the principles of data flow analysis and program transformation. The implementation uses a single-pass backward data flow analysis within basic blocks, making it efficient while maintaining correctness.

A. Contributions

This work makes the following contributions:

- A complete DSE implementation using backward data flow analysis
- Formal correctness proof with four invariants
- Comprehensive test suite including volatile store protection
- Automated benchmarking infrastructure demonstrating 10.84% improvement
- Production-ready CI/CD pipeline with quality gates

II. Related Work

Dead code elimination has been extensively studied in compiler optimization literature. Kennedy and Allen [1] provide comprehensive coverage of data flow analysis techniques. Aho et al. [2] describe the theoretical foundations of reaching definitions and liveness analysis that underpin our approach.

LLVM's built-in DSE pass uses more sophisticated techniques including alias analysis and inter-procedural optimization. Our implementation focuses on intra-procedural, basic-block-level optimization with conservative assumptions to ensure correctness.

III. Algorithm Design

A. Overview

Our DSE pass implements a backward data flow analysis to identify dead store instructions. The algorithm maintains a "live set" of pointer values whose contents may be read in the future, processing instructions in reverse order from bottom to top of each basic block.

B. Algorithm Phases

1) Phase 1: Conservative Initialization: Before backward traversal, we conservatively initialize the live set by assuming all pointer operands used in the block are potentially live at block exit. This ensures correctness when values escape the current basic block, pointers are passed to function calls, or control flow merges with other blocks.

```
let mut live_set: HashSet<PointerValue> =
    HashSet::new();
// Collect all pointer operands
let mut instr_iter =
    block.get_first_instruction();
while let Some(instr) = instr_iter {
    for i in 0..instr.get_num_operands() {
        if let Some(Operand::Value(val)) =
            instr.get_operand(i)
            && val.is_pointer_value() {
                live_set.insert(
                    val.into_pointer_value()
                );
            }
    }
```

```
    instr_iter = instr.get_next_instruction();
}
```

Conservative Initialization (src/analysis.rs:81-92)

2) Phase 2: Backward Traversal: The algorithm processes instructions in reverse order. For each instruction type:

Store Instructions:

1) Check if the store is volatile - if so, mark pointer as live and skip
2) Check if pointer is in live set:
   - If YES: Store is LIVE (provides a value that will be read), remove pointer from live set
   - If NO: Store is DEAD (value never read), mark for removal

Load Instructions: Add pointer operand to live set (creates demand for stored value)

Other Instructions: Conservatively add all pointer operands to live set

```
InstructionOpcode::Store => {
    // CRITICAL: Never eliminate volatile
    if instr.get_volatile().unwrap_or(false) {
        // Volatile - mark as live
        if let Some(Operand::Value(val)) =
            instr.get_operand(1)
            && val.is_pointer_value() {
                live_set.insert(
                    val.into_pointer_value()
                );
            }
        // Skip without marking as dead
    } else {
        // Non-volatile: check liveness
        // ...
    }
}
```

Volatile Store Protection (src/analysis.rs:100-108)

C. Transformation Phase

After analysis, the transformation pass removes all identified dead instructions from the module using LLVM's instruction removal API.

IV. Correctness Proof

A. Correctness Invariants

Theorem: The DSE pass preserves program semantics (observational equivalence).

Proof Sketch:

Invariant 1 (Load-Store Dependencies): If a load instruction reads from pointer P, all stores to P that may provide the loaded value are preserved.

Proof: When processing a load to P, we add P to the live_set. Any store to P encountered during backward traversal will find P in the live_set and will NOT be marked as dead. □

Invariant 2 (Volatile Store Protection): Volatile stores are never eliminated, regardless of liveness.

Proof: The algorithm explicitly checks `instr.get_volatile()` before any liveness analysis. Volatile stores are immediately marked as live and skipped. See src/analysis.rs:100-108. □

Invariant 3 (Conservative Analysis): When in doubt, the algorithm errs on the side of safety (preserving stores).

Proof:

- Phase 1 conservatively initializes all pointer operands as live
- Unknown instructions conservatively mark their pointer operands as live
- Inter-procedural effects are handled conservatively (all escaping pointers are live)

□

Invariant 4 (Side Effect Preservation): Stores with observable side effects are preserved.

Proof:

- Volatile: Explicitly protected (Invariant 2)
- Escaping pointers: Conservative initialization marks them live (Invariant 3)
- Atomic: Not currently handled (noted limitation)

□

B. Liveness Property

Definition: A pointer P is "live" at program point I if there exists a path from I to a load from P that does not overwrite P.

Lemma: The backward analysis correctly computes the live set at each instruction.

Proof by induction on instruction position (bottom-up):

Base case: At block exit, live_set contains all potentially-live pointers (conservative initialization).

Inductive step: Assume live_set is correct after instruction I. When processing I:

- If I is `load P`: P becomes live (correct by definition)
- If I is `store P` and P is live: A future load needs this value (correct)
- If I is `store P` and P is not live: No future load will read this value (correct to eliminate)
- Otherwise: Conservatively maintain liveness (correct)

□

V. Implementation Details

A. Technology Stack

- **Language:** Rust 2024 Edition
- **LLVM Bindings:** inkwell 0.7.0 (llvm21-1)
- **Build System:** Cargo
- **Target LLVM:** 21.1.6

B. Module Architecture

The implementation consists of four main modules:

lib.rs: Module declarations and comprehensive documentation

main.rs: CLI driver using clap for argument parsing (`-i` input, `-o` output)

TABLE I
BENCHMARK RESULTS

Metric	Before	After	Reduction
Store Instructions	83	74	9 (10.84%)
Total IR Lines	663	654	9 (1.36%)

analysis.rs: DSEAnalysis structure implementing backward data flow analysis with volatile protection

transform.rs: DSETransform implementing instruction removal

C. Key Data Structures

HashSet¡PointerValue¿: Efficiently tracks the live set during backward traversal

Vec¡InstructionValue¿: Accumulates dead instructions for later removal

BasicBlock: LLVM's representation of a basic block, providing instruction iteration

VI. EXPERIMENTAL EVALUATION

A. Test Suite

The implementation includes three categories of integration tests:

Test 1 - Optimization Capability (dse_target.ll):

- Input: 2 stores to same location (first is dead)
- Output: 1 store (50% reduction)
- Result: PASS

Test 2 - Semantic Preservation (no_op.ll):

- Input: 2 stores to same location (both live due to intermediate load)
- Output: 2 stores preserved (0% reduction)
- Result: PASS

Test 3 - Volatile Protection (volatile_protect.ll):

- Input: 8 stores (5 volatile, 3 non-volatile)
- Output: All 5 volatile stores preserved
- Result: PASS

B. Benchmark Results

We developed a comprehensive benchmark suite (benches/benchmark.c) containing 171 lines of C code with five test functions demonstrating various dead store patterns:

1) Simple dead stores (immediate overwrites)
2) Multiple sequential dead stores
3) Loop-invariant dead stores
4) Complex patterns with control flow
5) Memory-intensive patterns with cache arrays

The benchmark was compiled to LLVM IR using `clang -O1 -Xclang -disable-llvm-passes` and processed through our DSE pass.

Semantic Verification: The original and optimized binaries produce identical output, confirming correctness.

C. Performance Analysis

Eliminated Store Patterns:

- Redundant initializations: `int x = 0; x = value;`
- Overwritten temporaries: `temp = A; temp = B;`
- Dead loop stores: Values overwritten in each iteration

Correctly Preserved Patterns:

- Volatile stores: All preserved per specification
- Live stores: Values read by subsequent loads
- Last stores: Values potentially used after block exit

VII. LIMITATIONS AND FUTURE WORK

A. Current Limitations

Intra-Procedural Analysis Only: Analysis is limited to single basic blocks. No inter-procedural or inter-block optimization is performed.

Alias Analysis: Conservative: assumes distinct pointers may alias. Could eliminate more stores with precise alias analysis.

Atomic Operations: Atomic stores not explicitly handled. Should be treated similarly to volatile stores.

Memory Effects: Function calls conservatively mark all pointers as live. Could improve with function effect annotations.

B. Future Enhancements

Global Analysis: Extend to inter-procedural DSE using control flow graphs for whole-function analysis.

Alias Analysis Integration: Integrate with LLVM's alias analysis to enable more aggressive elimination with proven non-aliasing.

Atomic Handling: Add explicit checks for atomic stores and preserve all atomic operations.

Performance Metrics: Add execution time measurements, measure cache miss reduction, and quantify memory bandwidth savings.

Loop Optimizations: Detect and eliminate loop-invariant dead stores, handle induction variable patterns.

VIII. CI/CD INFRASTRUCTURE

To ensure code quality and reproducibility, we implemented a comprehensive GitHub Actions CI/CD pipeline with 8 parallel jobs:

1) **Code Quality:** Formatting validation and linting with strict warnings-as-errors
2) **Build & Test:** Multi-version testing on stable and nightly Rust with LLVM 21 auto-installation
3) **Benchmarking:** Automated benchmark execution with performance metric extraction and validation
4) **Documentation:** Rust doc generation and link validation
5) **Security Audit:** Dependency vulnerability scanning
6) **Code Coverage:** LCOV coverage report generation
7) **Release Build:** Optimized binary creation and packaging
8) **Pipeline Summary:** Aggregated status dashboard

The pipeline automatically runs on every push and pull request, providing immediate feedback on code quality and correctness.

IX. Conclusion

This work presents a complete implementation of Dead Store Elimination for LLVM IR using backward data flow analysis. The implementation demonstrates:

- **Correctness:** All tests pass with semantic equivalence verified
- **Effectiveness:** 10.84% store reduction on benchmark code
- **Safety:** Volatile stores and live values properly protected
- **Robustness:** Conservative analysis prevents incorrect elimination

The system achieves measurable performance improvements while maintaining semantic correctness through careful analysis design and comprehensive testing. The implementation provides a solid foundation for LLVM IR optimization and demonstrates the principles of compiler optimization in a production-ready system.

Future work will focus on extending the analysis to interprocedural optimization, integrating sophisticated alias analysis, and expanding coverage to handle atomic operations and complex control flow patterns.

Acknowledgment

The authors would like to thank the course instructors and teaching assistants of CSIE 5054 Advanced Compiler Design at National Taiwan University for their guidance and support.

References

[1] K. Kennedy and J. R. Allen, *Optimizing Compilers for Modern Architectures: A Dependence-based Approach*. Morgan Kaufmann, 2002.
[2] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2006.
[3] LLVM Project, "LLVM Language Reference Manual," https://llvm.org/docs/LangRef.html, 2025.
[4] "inkwell Documentation," https://docs.rs/inkwell/, 2025.
[5] K. D. Cooper and L. Torczon, *Engineering a Compiler*, 2nd ed. Morgan Kaufmann, 2011.